

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

**RESEARCH AND DEVELOPMENT OF A CODE RETRIEVAL
SYSTEM BASED ON HYBRID-RAG ARCHITECTURE COMBINING
KNOWLEDGE GRAPHS AND SEMANTIC SEARCH**

by

Tran Tien Duat

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

**RESEARCH AND DEVELOPMENT OF A CODE RETRIEVAL
SYSTEM BASED ON HYBRID-RAG ARCHITECTURE COMBINING
KNOWLEDGE GRAPHS AND SEMANTIC SEARCH**

by

Tran Tien Duat

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

Supervisor:

Dr Doan Xuan Huy Minh

© Copyright by Tran Tien Duat 2026

Research and Development of a Code Retrieval System Based on Hybrid-RAG Architecture Combining Knowledge Graphs and Semantic Search

Tran Tien Duat

Degree Master of Software Engineering

FPT University

2026

Abstract

Code querying is a complex problem because source code contains both static structural information (function calls, class inheritance, and module definitions) and semantic information. A vector-only Retrieval-Augmented Generation (RAG) index can represent semantic similarity, but it does not directly expose typed code relations for relation-heavy questions.

This thesis designs and evaluates a **Hybrid-RAG** architecture that combines an Abstract Syntax Tree (AST)-derived Knowledge Graph (KG) in FalkorDB with dense retrieval in Qdrant. The implementation uses query-aware graph traversal, vector search, weighted Reciprocal Rank Fusion (RRF) [2], and token-budgeted context assembly. Paired benchmarks across pinned LlamaIndex Core, Transformers, and LangChain Core workloads contain 230 source-reviewed questions and 1,380 generated and scored answers. Hybrid-RAG produced a statistically positive Faithfulness difference on Transformers (+0.169, 95% CI [0.043, 0.297]) and LangChain (+0.089 [0.017, 0.163]), while the LlamaIndex difference was inconclusive. Answer Correctness was statistically positive on LlamaIndex (+0.046 [0.002, 0.094]) and Transformers (+0.052 [0.002, 0.104]), but inconclusive on LangChain. No repository showed a conclusive Answer Relevancy difference. Hybrid retrieval added 182–237 ms of mean retrieval latency, while local generation remained the dominant cost at approximately 19–97 seconds. All embedding, generation, and judging calls used local Ollama services. The evidence supports replication across three Python repository workloads relative to vector-only retrieval, but not cross-language or alternative-baseline superiority.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr Doan Xuan Huy Minh, for the guidance, thoughtful feedback, and encouragement provided throughout this thesis. I also thank the lecturers and reviewers at FPT University for their comments and recommendations, which helped improve both the research and this report. Finally, I am grateful to my family, colleagues, and friends for their patience and support during the completion of this work.

Table of Contents

Acknowledgments	2
Table of Contents	3
List of Tables	5
List of Figures	6
List of Abbreviations	7
1 Introduction	1
1.1 System Context	1
1.2 Problem Statement and Bottlenecks	1
1.3 Related Works	2
1.4 Proposed Solution	3
1.5 Research Gap and Technical Contributions	3
1.6 Research Objectives & Questions	3
1.6.1 Research Objectives	3
1.6.2 Research Questions	4
1.7 Research Scope & Definition of Done	4
1.7.1 Research Scope	4
1.7.2 Definition of Done (DoD)	5
2 Theoretical Background and System Architecture	6
2.1 Theoretical Background & Literature Review	6
2.1.1 Information Retrieval (IR) Theory	6
2.1.2 Code Search Models	7
2.1.3 Retrieval-Augmented Generation (RAG) and Graph RAG	7
2.1.4 Literature Synthesis and Design Implications	8
2.2 System Architecture	9
2.3 System Requirements	9
2.4 High-level Architecture	9
2.5 Ingestion Pipeline	10
2.5.1 Incremental Indexing and Synchronization	11
2.5.2 Monorepo and Subdirectory Support	11

2.6	Hybrid Retrieval Flow	11
3	Detailed Design	13
3.1	Database Schema	13
3.2	Core Algorithms	14
3.2.1	Global Entity Resolution	14
3.2.2	Reciprocal Rank Fusion (RRF)	15
3.3	Community Detection and Architectural Clustering	16
3.3.1	Directory-Based Partitioning and Summarization	17
3.3.2	Integration with the Retrieval Pipeline	17
3.4	Multi-Repository Knowledge Graph and Visualization Portal	17
3.4.1	Multi-Repository Master Graph	18
3.4.2	Hierarchical Drill-Down Navigation	18
3.4.3	Performance Optimizations for Concurrent Indexing	19
3.5	Programming-Language Scalability	19
3.6	Software Testing	19
4	Experimental Results and Discussion	20
4.1	Test Environment	20
4.2	Metrics Justification	20
4.3	Evidence Provenance and Excluded Historical Diagnostic	21
4.4	Independent Answer-Quality Benchmark	21
4.5	Generalizability and Baseline Coverage	23
4.6	Architectural Optimization and Enhancements	23
4.7	Latency Breakdown	24
4.8	Case Study	25
5	Conclusion and Future Work	26
5.1	Conclusion	26
5.2	Limitations	27
5.3	Future Work	27

List of Tables

Table 1.	Comparison matrix of current code querying solutions	2
Table 2.	Implemented FalkorDB node labels and roles	13
Table 3.	Answer quality by repository with paired case-level 95% confidence intervals .	22
Table 4.	Repository and baseline coverage	23
Table 5.	Latency and token breakdown by repository	24

List of Figures

Figure 1.	Overall data flow diagram of the Hybrid-RAG system	9
Figure 2.	Data processing flow of the Ingestion Pipeline	10
Figure 3.	Concurrent graph–vector path for a generic local query	12
Figure 4.	Decision flow diagram of the Global Entity Resolution process	14
Figure 5.	Processing flow diagram of the Reciprocal Rank Fusion (RRF) algorithm	16
Figure 6.	Hierarchical navigation model of the multi-repository graph portal	18

List of Abbreviations

AST Abstract Syntax Tree. 1

FQN Fully Qualified Name. 10

KG Knowledge Graph. 1

LLM Large Language Model. 1

RAG Retrieval-Augmented Generation. 1

RRF Reciprocal Rank Fusion. 1, 2, 3, 15

SLA Service Level Agreement. 9

1 Introduction

1.1 System Context

Modern software maintenance requires developers to navigate large, evolving codebases and recover relationships that are distributed across files and modules. Large Language Models (LLMs) create new opportunities for programming assistance [8]; however, placing an entire repository in one prompt is constrained by finite context budgets and by degraded use of information located in the middle of long contexts [6].

The RAG technique [1] addresses this problem by retrieving a bounded set of source evidence and inserting it into the LLM context. A vector-only implementation can retrieve semantically related code, but its index does not explicitly represent call, import, definition, or inheritance edges. This motivates testing whether an AST-derived graph channel adds useful evidence beyond the same system operated in vector-only mode.

1.2 Problem Statement and Bottlenecks

The research problem is to retrieve evidence for questions that combine semantic intent with repository-specific structure while keeping the source and model calls local. Four gaps shape the work:

- **Representation gap:** dense chunks encode semantic similarity but not explicit repository topology.
- **Context constraint:** source evidence must be selected and serialized within a bounded prompt budget.
- **Index integrity:** repository identity, source revision, schema, and embedding model must remain consistent across graph and vector stores.
- **Evidence gap:** repository-specific point estimates are insufficient for broad claims without paired uncertainty estimates, replicated workloads, and stronger baselines.

1.3 Related Works

To solve the code querying problem, existing technologies exhibit various strengths and limitations.

Table 1 presents a comparison matrix of the solutions:

Table 1. Comparison matrix of current code querying solutions

Solution	Core Mechanism	Role in this thesis	Main trade-off
BM25 / lexical retrieval [3]	Exact-term ranking	Literature baseline; not executed	Strong symbol matching / weak paraphrase handling.
Vector-only RAG [1]	Dense chunks + cosine similarity	Executed baseline	Semantic matching / no explicit topology.
Code encoders (CodeBERT [9], Graph-CodeBERT [13])	Pre-trained code representation	Literature comparison; not executed	Learned code semantics / requires model-specific indexing.
Microsoft GraphRAG [4]	Entity graph + community summaries	Architectural comparison; not executed	Global synthesis / additional indexing and summarization work.
This prototype	AST graph + dense retrieval + weighted RRF	Evaluated against vector-only	Structural and semantic evidence / higher retrieval latency and indexing complexity.

The comparison separates related approaches from executed baselines. Only vector-only retrieval is run under the same source snapshot, models, questions, and repeated-measure protocol. BM25, graph-only retrieval, code encoders, and Microsoft GraphRAG remain necessary baselines for a broader comparative study.

1.4 Proposed Solution

We propose a query-aware Hybrid-RAG solution using FalkorDB [15] and Qdrant [16]. Exact relation questions use deterministic graph traversal when a relation target is found; other local questions can search graph and vector channels concurrently and fuse their rankings with weighted RRF; global questions retrieve repository-scoped community summaries. The selected evidence is assembled under a token or character budget before local generation. Chapter 2 derives this design from the literature, and Chapter 3 maps it to the implementation.

1.5 Research Gap and Technical Contributions

Hybrid retrieval, AST parsing, graph retrieval, and RRF are established techniques. The thesis therefore does not claim a new retrieval algorithm. Its contribution is an evidence-backed system design and integration for private codebase understanding:

- a repository-scoped dual index that maps Python AST entities and typed relations to FalkorDB while storing source-aligned semantic chunks in Qdrant;
- query-aware retrieval that combines deterministic relation traversal, concurrent graph–vector retrieval, weighted rank fusion, sibling expansion, and budgeted context assembly;
- integrity controls including conservative entity resolution, replacement-before-delete index updates, source/schema/model provenance validation, cache-generation invalidation, and a local-only Ollama boundary; and
- reproducible paired answer-quality benchmarks across three pinned Python repository workloads, with raw records, case-level confidence intervals, and explicit limits on corpus, language, and baseline coverage.

1.6 Research Objectives & Questions

1.6.1 Research Objectives

The primary objective is to determine whether adding AST-derived structural retrieval to an otherwise identical vector-RAG pipeline improves grounded answers across multiple Python repository

workloads under local execution constraints. The specific objectives are:

- implement a repository-scoped graph–vector retrieval pipeline with reproducible source provenance and bounded context assembly;
- compare Hybrid-RAG with vector-only retrieval using matched questions, pinned source identities, models, and repeated runs within each repository; and
- quantify answer quality, retrieval/generation latency, and token usage with paired confidence intervals while documenting threats to validity.

1.6.2 Research Questions

The thesis addresses three research questions aligned with the available measurements:

- **RQ1 — Answer quality:** Across the pinned LlamaIndex, Transformers, and LangChain workloads, how does Hybrid-RAG change Faithfulness, Answer Relevancy, and Answer Correctness relative to vector-only retrieval?
- **RQ2 — Efficiency trade-off:** What changes in retrieval latency, local generation latency, and prompt/completion token counts accompany Hybrid-RAG?
- **RQ3 — Evidence strength:** Which observed differences remain above or below zero under paired case-level 95% confidence intervals, and what limits their generalization?

1.7 Research Scope & Definition of Done

1.7.1 Research Scope

The evaluated implementation is scoped to Python. Java indexing is explicitly rejected because a Java entity extractor and resolver are not implemented, even though a Java grammar is used by the separate context skeletonizer. The completed answer-quality evidence covers pinned LlamaIndex Core, Transformers, and LangChain Core workloads and one executed baseline (vector-only). It uses `nomic-embed-text` for embeddings, `gemma4:12b` for answer generation, and `qwen2.5-coder:7b` as an independent local judge. The LlamaIndex workload contains 30 questions; the Transformers and LangChain workloads contain 100 questions each. Their indexed

corpus scopes are not identical, so results are reported per repository rather than pooled. Results for Kubernetes, BM25, graph-only retrieval, or Microsoft GraphRAG are not available and are not inferred.

1.7.2 Definition of Done (DoD)

The evaluated prototype uses the following evidence criteria:

1. persist complete raw records for both retrieval modes under a pinned source identity and index schema;
2. report Faithfulness, Answer Relevancy, and Answer Correctness with paired case-level 95% confidence intervals;
3. report retrieval latency separately from local generation latency and token counts; and
4. treat pooled cross-repository, cross-language, and additional-baseline conclusions as unverified until matched dedicated experiments are completed.

2 Theoretical Background and System Architecture

2.1 Theoretical Background & Literature Review

2.1.1 Information Retrieval (IR) Theory

Information Retrieval (IR) is the process of obtaining information resources relevant to an information need from a collection of resources. In modern search pipelines, two primary IR paradigms dominate:

- **Lexical Retrieval (Keyword Matching):** Classical retrieval algorithms like TF-IDF and BM25 [3] match query terms against document terms. The BM25 score of a document D for a query Q with terms q_i is computed as:

$$\text{Score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (2.1)$$

where $f(q_i, D)$ is the term frequency in the document, $|D|$ is the document length, and avgdl is the average document length. BM25 is highly effective at finding exact class names, method signatures, or variables in codebases. However, it fails when queries use synonyms or semantic descriptions that do not match exact characters.

- **Dense Semantic Retrieval:** Dense retrieval utilizes neural embedding models to convert text into high-dimensional vector spaces. Semantic similarity between query vector $\mathbf{u}$ and document vector $\mathbf{v}$ is computed using Cosine Similarity:

$$\text{Similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (2.2)$$

While dense retrieval captures abstract user intent, it is susceptible to noise and ignores the strict lexical syntax and structural morphology unique to software codebases.

2.1.2 Code Search Models

Source code fundamentally differs from natural language due to its strict structural syntax and logic constraints. Code representation models have evolved through three generations:

1. **Text-based Representation:** Treating code as plain text, utilizing BM25 or regex searches. This completely discards programming grammar.
2. **Deep Pre-trained Language Models:** Models such as CodeBERT [9] and GraphCodeBERT [13] learn code representations from large corpora; GraphCodeBERT additionally incorporates data-flow structure. They operate on bounded inputs and do not by themselves maintain an explicit, updateable graph of repository-specific relationships.
3. **Static Dependency Graphs:** Leveraging language parsers to build static relationship maps. Formally, we define the codebase knowledge graph as a directed multigraph $G = (V, E, \phi, \psi)$, where:
 - V is the set of emitted AST code entity nodes (Module, Class, Function).
 - E is the set of directed edges representing syntax relations.
 - $\phi : V \rightarrow L_V$ is the node labeling function mapping vertices to $L_V = \{\text{Module, Class, Function}\}$.
 - $\psi : E \rightarrow L_E$ is the edge labeling function mapping connections to syntax dependencies $L_E = \{\text{DEFINES, CALLS, INHERITS, IMPORTS}\}$.

2.1.3 Retrieval-Augmented Generation (RAG) and Graph RAG

Retrieval-Augmented Generation (RAG) [1] optimizes LLM performance by injecting external context into the prompt. The paradigm has evolved as follows:

- **Naïve RAG:** Chunks source code at fixed text boundaries. Such windows can split functions or classes and lose structural context.
- **Advanced RAG:** Introduces syntax-aware chunking based on AST boundaries (Class/Function scopes) and post-retrieval reranking.

- **Graph RAG:** Incorporates a knowledge-graph index. Frameworks such as Microsoft GraphRAG [4] use entity extraction and hierarchical community summaries for global questions. This adds an indexing and summarization stage whose quality, latency, and compute cost depend on the selected models and corpus.

To combine the benefits of both dense semantic and structural retrieval, we propose a Hybrid-RAG system coordinated via Reciprocal Rank Fusion (RRF) [2]:

$$RRF_Score(d \in D) = \sum_{m \in M} \frac{w_m}{k + r_m(d)} \quad (2.3)$$

where $M = \{\text{vector}, \text{graph}\}$, $r_m(d)$ is the rank of document d in retrieval stream m , w_m is the weight of the retrieval stream, and k is a smoothing parameter set to 60 in this implementation. Rank fusion avoids assuming that vector similarities and graph scores share a calibrated numerical scale.

2.1.4 Literature Synthesis and Design Implications

The reviewed approaches are complementary rather than interchangeable. Lexical retrieval is useful for exact identifiers; dense retrieval covers paraphrased intent; AST and dependency graphs expose typed relations; and RAG constrains generation to selected evidence. This synthesis leads to five design decisions: (i) parse code at symbol boundaries instead of fixed character windows, (ii) store semantic chunks and typed graph relations in separate indexes, (iii) route explicit relation questions to deterministic graph plans, (iv) use rank fusion when both channels contribute candidates, and (v) assemble the final evidence under a strict context budget with source metadata. The implementation evaluates the incremental value of the graph channel against vector-only retrieval. It does not evaluate every approach in the literature, so BM25, graph-only, learned code encoders, and Microsoft GraphRAG remain comparison gaps rather than measured inferiors.

2.2 System Architecture

2.3 System Requirements

- **Functional Requirements:** The system must support static source code parsing to construct relationship graphs, semantic search based on cosine similarity, hybrid querying (combining graph structures and text), and return answers alongside precise code references.
- **Non-functional Requirements (Service Level Agreement (SLA)):** Retrieval response latency $< 1s$ as a retrieval target, concurrency support during embedding generation, and a local-only model-provider boundary.

2.4 High-level Architecture

The Hybrid-RAG system is designed following the Hexagonal architecture (Ports and Adapters) [14] to decouple physical database infrastructure layers from the core search engine business logic. The overall architecture and data flow are described in Figure 1.

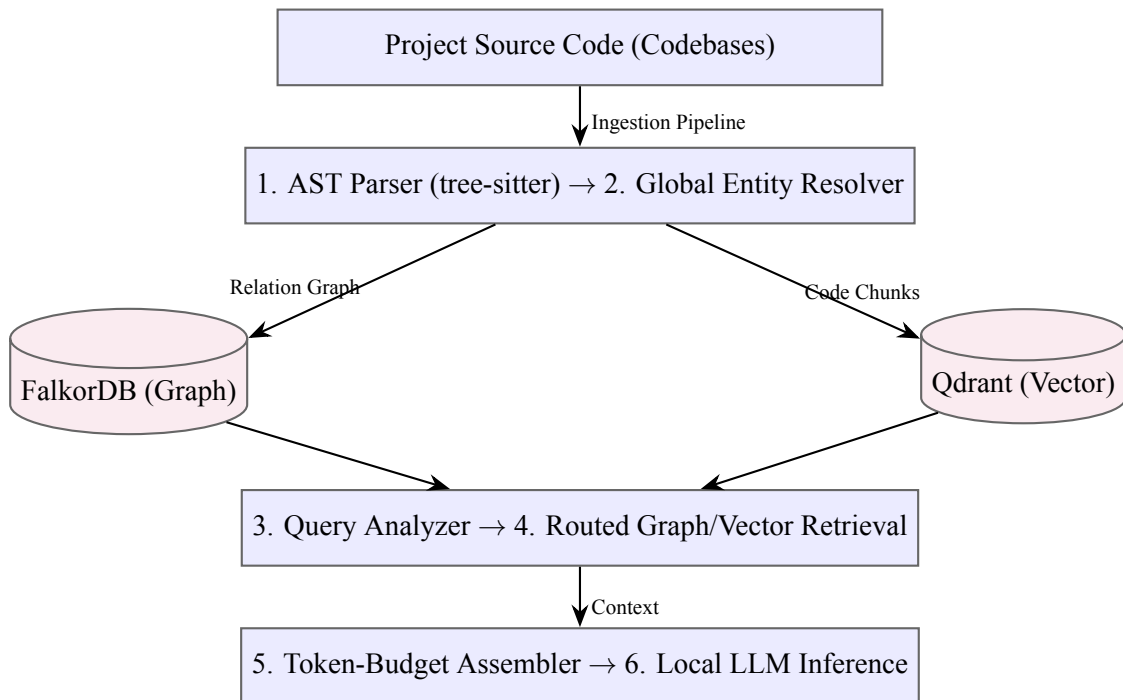


Figure 1. Overall data flow diagram of the Hybrid-RAG system

2.5 Ingestion Pipeline

The ingestion pipeline converts raw source code into structural relationship graphs and semantic embedding vectors. This process consists of 4 main phases, as illustrated in Figure 2:

1. **Phase 1 — AST Parsing:** The Tree-Sitter library [5] parses Python source code into Abstract Syntax Trees. The system extracts entities (Module, Class, Function) and automatically assigns dot-notated Fully Qualified Name (FQN) strings (`package.module.Class.method`) to ensure uniqueness across the repository. Java projects are rejected at the indexing boundary until a Java extractor is implemented.
2. **Phase 2 — Global Entity Resolution:** Unresolved call stubs are resolved conservatively using caller context: (i) methods in the same class, (ii) module-level functions in the caller's module, and (iii) import-aware unique candidates. Remaining stubs are matched in one batch against previously indexed graph nodes and redirected only when exactly one target exists. The algorithm details are described in Section 3.2.1.
3. **Phase 3 — Source Code Chunking:** Source code is chunked at AST function and class boundaries instead of fixed character windows, preserving symbol-level boundaries where parsing succeeds.
4. **Phase 4 — Parallel Embedding:** Chunks are embedded through a bounded ThreadPool Executor; the default `EMBED_CONCURRENCY` is 8. The `nomic-embed-text` model [17] generates 768-dimensional vectors through local Ollama [18]. No controlled sequential-versus-parallel embedding benchmark is reported in this thesis.

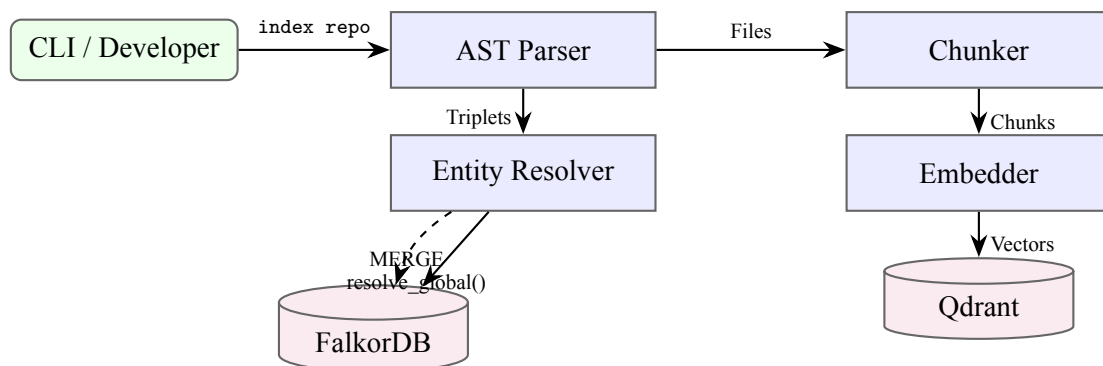


Figure 2. Data processing flow of the Ingestion Pipeline

2.5.1 Incremental Indexing and Synchronization

To prevent parsing and embedding the entire repository from scratch upon every codebase change (which is CPU/GPU and time intensive), the system implements an incremental indexing and synchronization mechanism:

- **Commit Hash Tracking:** The system persists the last indexed commit hash for each repository directly inside the FalkorDB graph database.
- **Change Detection:** During re-indexing, the system leverages Git diff mappings to isolate modified, added, renamed, or deleted files.
- **Targeted Cleanup:** Before processing new changes, the system executes targeted queries to purge the old AST entities of modified/deleted files from FalkorDB and deletes their corresponding vector points from Qdrant.
- **Delta Processing:** Only newly added or modified files are passed through the ingestion pipeline. The implementation reduces repeated work by construction, but this thesis does not report a controlled percentage reduction for incremental indexing.

2.5.2 Monorepo and Subdirectory Support

To meet modern industrial software engineering requirements, the ingestion pipeline implements a recursive upward Git repository resolution algorithm. In multi-package monorepos, codebase paths defined in the configurations often target specific nested package subdirectories rather than the root directory. By recursively traversing the file tree upwards from the configured subdirectory path until it detects a valid `.git` folder, the system correctly identifies and resolves the monorepo root. This ensures that change detection, commit tracking, and incremental diff synchronization function correctly and transparently across individual monorepo components without requiring independent Git setups.

2.6 Hybrid Retrieval Flow

The query analyzer selects among three implemented paths. Global questions return repository-filtered community summaries. Explicit relation questions first execute a deterministic graph plan

and fall back to vector retrieval only when no relation target is found. Generic local questions search the graph and vector channels concurrently and fuse their ranks. Figure 3 shows this third, generic-local path:

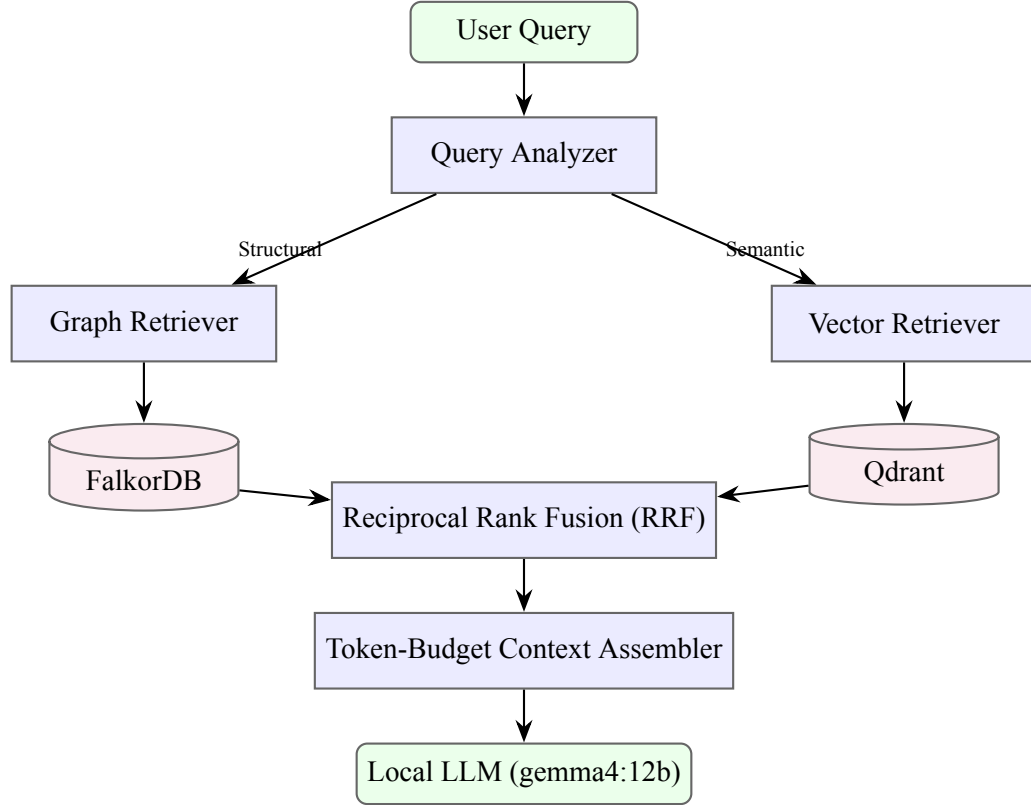


Figure 3. Concurrent graph–vector path for a generic local query

The figure is not a universal control-flow diagram: global community retrieval and successful exact-relation traversal bypass the parallel fusion branch. This routing avoids paying for or diluting a deterministic graph answer when the query names a supported relation.

3 Detailed Design

3.1 Database Schema

The system stores parallel data structures in two databases:

1. **Static Relationship Graph (FalkorDB [15]):** The graph structure including node labels and relation types is detailed in Table 2.
2. **Vector Fragments (Qdrant [16]):** Stores 768-dimensional vectors generated from nomic-embed-text alongside a metadata payload including: `node_id`, `file_path`, `text_content`, and `repository` (used for repository namespace filtering).

Table 2. Implemented FalkorDB node labels and roles

Node label	Representative properties	Implemented role
Repository	<code>id</code> , <code>repository</code> ,	Repository identity and synchronization state.
Metadata	<code>last_indexed_commit</code>	
Directory	<code>id</code> , <code>name</code> , <code>path</code> , <code>repository</code>	File-tree hierarchy and module ownership.
Module	<code>id</code> , <code>name</code> , <code>file_path</code> , <code>repository</code> , <code>type</code>	Python source file and import origin.
Class	<code>id</code> , <code>name</code> , <code>file_path</code> , <code>repository</code>	Python class definition and inheritance source.
Function	<code>id</code> , <code>name</code> , <code>file_path</code> , <code>signature</code> , <code>repository</code>	Function or method definition and call source.
Document/ Configuration	<code>id</code> , <code>name</code> , <code>file_path</code> , <code>repository</code> , <code>text</code>	Optional generic Markdown, YAML, and Dockerfile content.
Community	<code>id</code> , <code>name</code> , <code>summary</code> , <code>level</code>	Directory-based architectural summary.

The implemented relationship vocabulary includes `DEFINES`, `IMPORTS`, `INHERITS`, `CALLS`, `USES`, `DEFINED_IN`, `WEAK_LINK`, `IN_COMMUNITY`, and `COMMUNITY_DEPENDS`. `Variable` is reserved as a compatibility target for optional LLM-extracted `USES` relations; the current AST parser does not emit `Variable` nodes.

3.2 Core Algorithms

This section examines two implementation mechanisms that materially affect relationship correctness and retrieval ranking:

3.2.1 Global Entity Resolution

When a parsed call target cannot be bound immediately, the parser stores an explicit stub such as `__call__foo`. The resolver uses caller context first and leaves ambiguous targets unresolved instead of creating an arbitrary edge. Figure 4 summarizes the decision order.

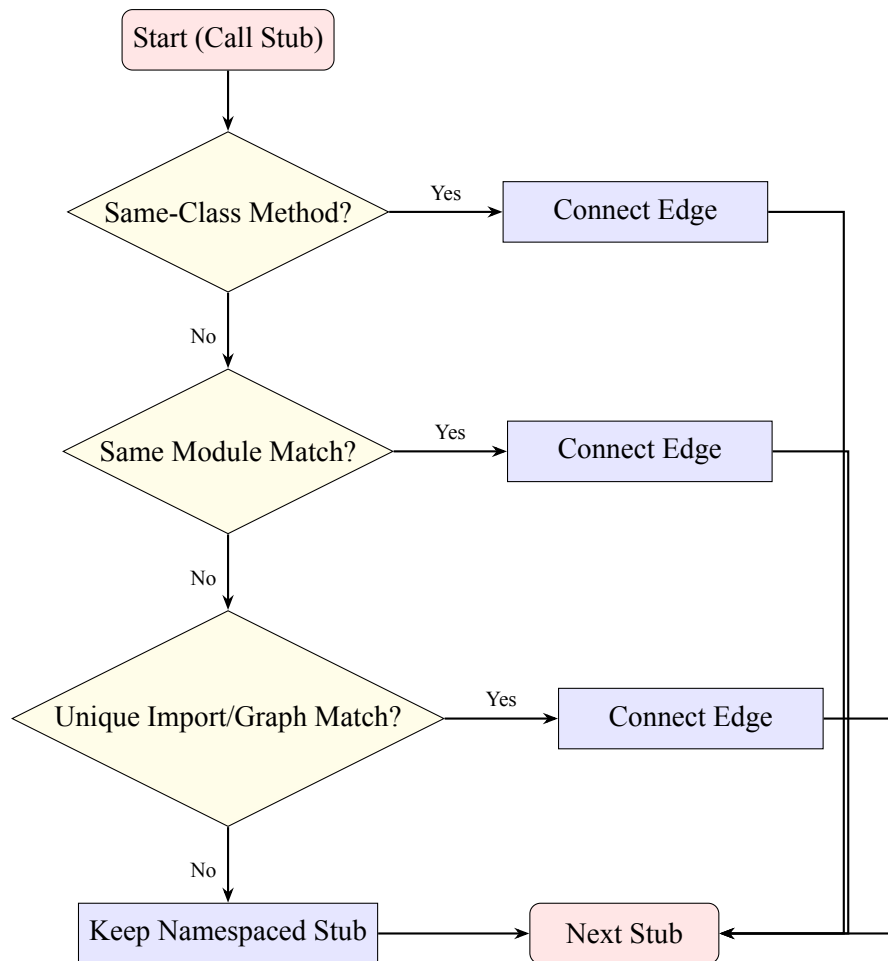


Figure 4. Decision flow diagram of the Global Entity Resolution process

The specific steps in the resolution flow are:

1. **Same-Class Method Check:** For a bare, `self.`, or `cls.` call inside a class, the resolver

checks for the corresponding method in that same class.

2. **Same-Module Function Check:** For an unresolved bare call, the resolver checks for a matching module-level function in the caller’s module.
3. **Import-Aware and Existing-Graph Check:** The local resolver accepts an imported function only when the import context identifies one candidate. Remaining stubs are queried against previously indexed graph nodes in one batch and are redirected only when exactly one target exists. Ambiguous or missing matches remain explicit and are namespaced by repository.

3.2.2 Reciprocal Rank Fusion (RRF)

When a retrieval path returns one or two ranked lists, the RRF algorithm [2] computes integrated rank scores via:

$$\text{score}(d) = \sum_{r \in R} \frac{W_r}{k + \text{rank}_r(d)} \quad (3.1)$$

where $k = 60$ is the default smoothing parameter and W_r is the weight for retrieval source r . With the checked-in configuration, a successful exact-relation path uses graph-only fusion with configured graph weight 3.0. Generic local fusion caps the graph contribution at 1.0 and uses vector weight 1.0 so noisy neighborhood expansion cannot overpower semantic ranking; vector-only mode uses weights (0, 1). The legacy `hybrid_weight` setting is retained for compatibility but is not applied by the current retriever. Figure 5 illustrates the two-list accumulation mechanism.

RRF was selected instead of adding graph and vector scores directly because the two retrieval channels do not expose scores on a guaranteed common scale. Direct weighted addition would therefore require an additional normalization and calibration rule before the weights could be interpreted consistently. Rank fusion avoids that cross-channel scale assumption, while the configured weights still express the intended priority of each channel. A normalized weighted-sum fusion is a reasonable alternative, but it was not implemented as a controlled baseline in this thesis; the results therefore do not claim that RRF is universally superior.

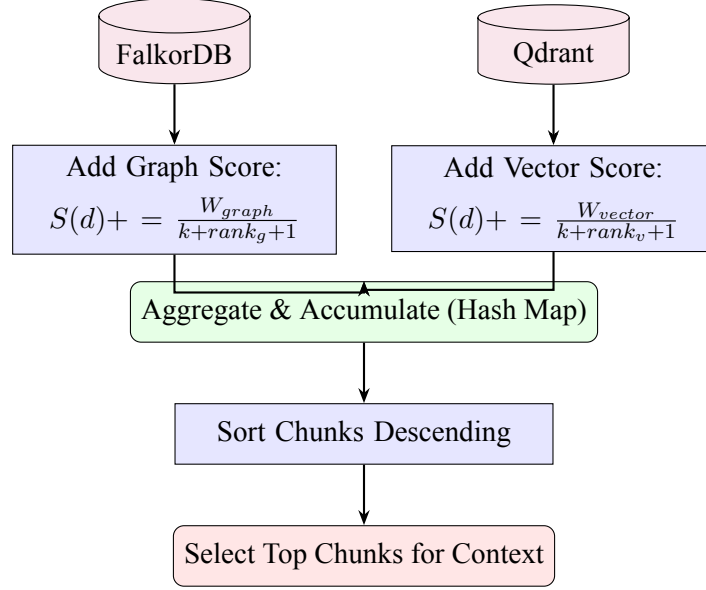


Figure 5. Processing flow diagram of the Reciprocal Rank Fusion (RRF) algorithm

The steps involved in the RRF scoring flow are:

1. **Initialize Score Map:** Create an empty hash map to store accumulated fusion scores for each candidate document/code chunk.
2. **Accumulate Graph Retrieval Scores:** Iterate through FalkorDB query results. For the i -th chunk (0-indexed rank), add a score increment of $W_{graph}/(k+i+1)$ to its map entry.
3. **Accumulate Vector Retrieval Scores:** Iterate through Qdrant query results. For the j -th chunk, add a score increment of $W_{vector}/(k+j+1)$ to its map entry.
4. **Sort and Filter:** Sort candidate chunks by their accumulated score in descending order, returning the highest-scoring candidates for context assembly.

3.3 Community Detection and Architectural Clustering

A key limitation of low-level graph traversal is that it exposes calls, imports, and inheritance without directly presenting a repository-level architectural summary. The implemented community builder addresses this by grouping entities into repository-scoped directory communities and generating a local summary for each group. API background indexing invokes this step as non-fatal post-processing; the CLI also exposes an explicit `community-build` command.

3.3.1 Directory-Based Partitioning and Summarization

The final implementation does not use Louvain clustering. An earlier design considered modularity-based communities, but the codebase replaced that approach with deterministic directory partitioning:

1. **Graph projection:** code nodes and typed edges are read from FalkorDB; existing `Community` nodes are excluded.
2. **Repository and directory grouping:** each code node is assigned by the pair `(repository, dirname(file_path))`, preventing same-named directories in different repositories from being merged.
3. **Local summary generation:** entity lists, internal relations, and boundary relations are serialized and sent to the configured local Ollama model to generate a community title and architectural summary.
4. **Persistence:** `Community`, `IN_COMMUNITY`, and weighted `COMMUNITY_DEPENDS` relationships are written back to FalkorDB.

3.3.2 Integration with the Retrieval Pipeline

Community nodes serve two purposes:

- **Architectural Navigation:** The interactive visualization portal exposes community nodes as the first drill-down level when a user enters a repository, giving an immediate high-level view of the codebase's logical subsystems.
- **Global-query context:** global retrieval reads repository-filtered community summaries and supplies them to the local answer-generation path.

3.4 Multi-Repository Knowledge Graph and Visualization Portal

To support engineering workflows involving multiple interdependent codebases (microservices, monorepos, shared libraries), the system implements a **multi-repository knowledge graph** and a corresponding interactive visualization portal accessible via a web browser.

3.4.1 Multi-Repository Master Graph

A dedicated endpoint (GET /graph/master) constructs a global graph view spanning all indexed repositories:

- Each repository is represented by a `RepositoryMetadata` node.
- Directory nodes are automatically generated from parsed source file paths — including repositories that consist entirely of Python code files, ensuring structural coverage regardless of the presence of non-code files (`.yaml`, `.md`).
- Noise directories (`node_modules`, `.venv`, `.agents`, `__pycache__`, `.git`, etc.) are filtered at the Cypher query level, preventing stale legacy nodes from polluting the master graph.

3.4.2 Hierarchical Drill-Down Navigation

The portal implements a three-level navigation hierarchy, illustrated in Figure 6:

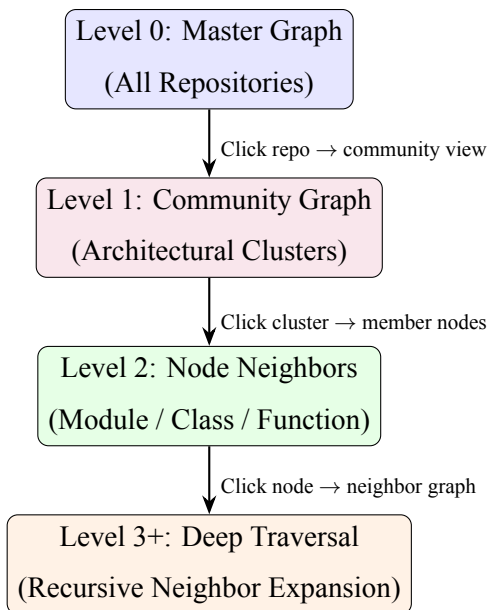


Figure 6. Hierarchical navigation model of the multi-repository graph portal

A **breadcrumb navigation bar** persists across all levels, rendering the complete drill-down path (e.g., Master ▷ langchain-core ▷ Retrieval Cluster ▷ BaseRetriever). Any intermediate node in the breadcrumb is clickable, allowing the user to jump back to any previously visited level without losing context.

3.4.3 Performance Optimizations for Concurrent Indexing

Two bounded-concurrency mechanisms are implemented:

- **Concurrent repository indexing:** API background jobs use an `asyncio.Semaphore` whose `INDEXING_CONCURRENCY` default is 3.
- **Parallel AST parsing:** files within one repository are parsed with a bounded `ThreadPoolExecutor`; the worker limit is derived from CPU count and capped at 32.

These mechanisms are present and unit-tested, but the thesis does not claim an end-to-end indexing speedup because no controlled sequential-versus-concurrent repository benchmark is persisted.

3.5 Programming-Language Scalability

The storage ports, repository namespaces, rank fusion, and context assembly are largely language-independent after canonical entities and chunks exist. The ingestion semantics are not. The production indexing boundary currently accepts only `languages=["python"]`; Java-only repositories are rejected, and the parser returns an explicit not-implemented error for Java extraction. Supporting another language requires a Tree-Sitter grammar, language-specific entity and relation extraction, canonical symbol resolution, chunk construction, and language-specific ground-truth tests. The existing Java-aware skeletonizer only formats already selected Java files and does not constitute Java indexing support. Kubernetes, whose principal implementation language is Go, therefore cannot be used as a comparable AST benchmark until a Go ingestion adapter is implemented.

3.6 Software Testing

The codebase is validated via 313 passing automated tests, with 27 explicitly skipped integration cases and two warnings in the final merged verification. The suite covers parsing and extraction logic, end-to-end ingestion pipelines, evaluation preflight, and functional API surfaces. The answer-quality benchmarks are separate controlled workloads totaling 1,380 generated and scored records and are not counted as unit-test coverage.

4 Experimental Results and Discussion

4.1 Test Environment

- **Hardware:** Apple Mac mini M4 developer machine; the benchmark records local execution but does not claim a hardware-independent latency.
- **Software:** Python 3.12, FalkorDB, Qdrant, and Ollama with local embedding and generation endpoints.
- **Indexed sources:** pinned `llama-index-core==0.14.21` with deterministic source digest `sha256:13b55bcf111885b7c2c9c07886b5656769041331c81c28bcf3cbfb87939ec509`; Transformers commit `0a2757da521a7a49b8143d9e0c938f08747d682e`; and LangChain `libs/core` at commit `51578289bb1f696a643e0740be1441039d8af8ce`. All use index schema version 3 and `nomic-embed-text`.
- **Answer-quality suites:** LlamaIndex contains 30 cases (10 simple, 10 medium, 10 hard); Transformers and LangChain each contain 100 cases (35 simple, 35 medium, 30 hard). Every suite evaluates two retrieval modes with three repeats and seed 42, producing 1,380 generated and scored answers in total.
- **Statistical unit:** the question case within each repository; the three repeats are averaged within each case before paired bootstrap resampling with 10,000 resamples.

4.2 Metrics Justification

We selected evaluation metrics based on: (i) suitability to the code retrieval task, and (ii) comparability with prior research.

- **Hit Rate @K (K=5):** Measures the frequency of relevant documents appearing in the top-K returned results. It is retained as a diagnostic metric definition because a developer often needs at least one correct code block to proceed, rather than all K blocks being correct [21]. However, no valid schema-v3 Hit Rate@5 result is used as confirmatory evidence in this thesis: the historical structural report was generated against schema-v2 and failed the current provenance check.

- **Faithfulness** (RAGAS [7]): Evaluates how loyal the generated answer is to the retrieved context, revealing LLM hallucinations.
- **Answer Relevancy** (RAGAS [7]): Measures how relevant the generated answer is to the user’s question, complementary to Faithfulness.
- **Answer Correctness** (RAGAS [7]): Compares the generated answer with the reviewed reference answer.
- **Paired 95% Confidence Interval**: Quantifies uncertainty in Hybrid-minus-vector differences by resampling question-level pairs with replacement within each repository. Repeats are averaged within a case to avoid treating them as independent questions.
- **Latency Breakdown**: Segmenting processing time by phase helps target bottlenecks.

BLEU and ROUGE were not selected as confirmatory metrics because lexical overlap does not directly test whether an answer is grounded in retrieved evidence or correct about code relationships.

4.3 Evidence Provenance and Excluded Historical Diagnostic

The repository also contains a historical 50-case structural report generated against a schema-v2 index. Its current status file explicitly records that it predates the schema-v3 identity repair and that its RAGAS field is null. Because it does not satisfy the present provenance preflight, this thesis does not use its Hit Rate values as current confirmatory evidence. The analysis below is restricted to the three completed schema-v3 answer-quality artifacts, whose source identity, index run, embedding model, raw records, and review grade are persisted. The Transformers index provenance identifies a controlled gold-sparse source tree, whereas LangChain identifies the full `libs/core` subtree; this corpus-scope difference precludes treating the repository estimates as one pooled experiment.

4.4 Independent Answer-Quality Benchmark

We separately measured generated-answer quality with the RAGAS metrics [7]. Each repository workload evaluates vector-only and Hybrid-RAG modes and repeats each case three times with seed 42. The answer model is local `gemma4:12b`; an independent local `qwen2.5-coder:7b` judge

scores Faithfulness, Answer Relevancy, and Answer Correctness. LlamaIndex contributes 180 records, while Transformers and LangChain contribute 600 records each, for 1,380 generated and scored records.

The gold dataset is a manually maintained, checked-in artifact rather than a live dataset synchronized automatically with the latest repository release. Each case is reviewed against its pinned source snapshot; LlamaIndex records package/source digests, while the two Git-based workloads record immutable source commits and index provenance. A benchmark run is accepted only when these identities and the index provenance match; otherwise the run is rejected or treated as draft evidence. This makes the datasets auditable, but the manual maintenance process and AI source review remain limitations and do not replace independent human annotation.

Table 3. Answer quality by repository with paired case-level 95% confidence intervals

Repository	Metric	Vector mean [95% CI]	Hybrid mean [95% CI]	Δ [95% CI]
LlamaIndex	Faithfulness	0.896 [0.816, 0.956]	0.924 [0.843, 0.980]	+0.029 [−0.069, 0.115]
	Answer Relevancy	0.729 [0.618, 0.826]	0.808 [0.737, 0.859]	+0.079 [−0.0001, 0.175]
	Answer Correctness	0.652 [0.606, 0.703]	0.698 [0.650, 0.748]	+0.046 [0.002, 0.094]
Transformers	Faithfulness	0.441 [0.344, 0.538]	0.610 [0.514, 0.702]	+0.169 [0.043, 0.297]
	Answer Relevancy	0.716 [0.656, 0.772]	0.772 [0.719, 0.819]	+0.055 [−0.014, 0.126]
	Answer Correctness	0.574 [0.523, 0.625]	0.626 [0.582, 0.669]	+0.052 [0.002, 0.104]
LangChain	Faithfulness	0.856 [0.786, 0.920]	0.945 [0.900, 0.983]	+0.089 [0.017, 0.163]
	Answer Relevancy	0.854 [0.823, 0.877]	0.845 [0.818, 0.866]	−0.009 [−0.043, 0.026]
	Answer Correctness	0.699 [0.662, 0.734]	0.717 [0.679, 0.754]	+0.018 [−0.021, 0.057]

The paired intervals show a repository-dependent pattern. For LlamaIndex, only Answer Correctness excludes zero. For Transformers, Faithfulness and Answer Correctness exclude zero, while Answer Relevancy is inconclusive. For LangChain, only Faithfulness excludes zero; Answer Relevancy and Answer Correctness are inconclusive. Thus, Faithfulness replicates positively in the two new 100-case workloads, but no quality metric is conclusively positive in all three repositories. The source-hit rates are 1.00 in both LlamaIndex modes, 0.94 vector versus 0.97 hybrid for Transformers, and 1.00 vector versus 0.99 hybrid for LangChain. These source-hit fields are diagnostics and are not substituted for the excluded schema-v2 Hit Rate@5 result. The gold answers were approved by an AI source review, not independent human annotation; therefore the results are controlled local evidence rather than human-evaluated production accuracy rates.

4.5 Generalizability and Baseline Coverage

The requested replication on Transformers and LangChain is complete. Table 4 separates completed repository evidence from experiments that remain outstanding.

Table 4. Repository and baseline coverage

Target	Status	Interpretation
LlamaIndex Core	Completed	30 reviewed questions, 3 repeats, Hybrid and vector-only modes.
Transformers	Completed	100 reviewed questions, 3 repeats, pinned Git commit, and controlled gold-sparse index.
LangChain	Completed	100 reviewed questions, 3 repeats, pinned Git commit, and independent libs/core index.
Kubernetes	Not comparable yet	Primarily Go; the current AST ingestion pipeline is Python-only.
BM25 / graph-only / GraphRAG	Not executed	Required to distinguish hybrid gains from lexical, graph-only, and alternative graph-RAG designs.

Consequently, the benchmark answers RQ1–RQ3 separately for three pinned Python repository workloads. Replication across Transformers and LangChain is stronger evidence than a single-repository result, but the unequal case counts and corpus scopes do not support a pooled effect estimate. The experiment also does not establish generalization across programming languages or retrieval families.

4.6 Architectural Optimization and Enhancements

The implemented reliability improvements are evaluated as engineering controls rather than collapsed into one unsupported optimization percentage. When a repository is supplied as query scope, repository filtering prevents results from other namespaces; without that optional scope,

retrieval may search across indexed repositories. Index replacement writes the new graph and vector records before stale data is removed, reducing an empty-index window, but it does not provide reader-isolated generation cutover. Provenance checks require matching source identity, index run, schema, and embedding model; cache generations separate responses after indexing; and unsupported Java input is rejected before indexing. Optional YAML, Markdown, and Dockerfile ingestion is generic document/configuration chunking and is disabled by default; it does not provide Python-style AST semantics.

These controls make the retrieval and evaluation measurements reproducible. They should not be described as a single token-savings or accuracy percentage without a separate controlled experiment.

4.7 Latency Breakdown

The completed answer-quality benchmarks recorded retrieval, local generation, and token statistics for both retrieval modes. Table 5 reports case-level means and paired-bootstrap intervals per repository; these are not service-level percentiles.

Table 5. Latency and token breakdown by repository

Repository	Measure	Vector mean [95% CI]	Hybrid mean [95% CI]	Δ [95% CI]
LlamaIndex	Retrieval (ms)	134 [118, 151]	371 [348, 395]	+237 [208, 266]
	Generation (ms)	96,754 [84,413, 110,142]	87,268 [78,110, 96,465]	−9,486 [−23,036, 1,438]
	Prompt tokens	1,841 [1,633, 2,047]	1,740 [1,566, 1,913]	−101 [−282, 77]
	Completion tokens	934 [788, 1,092]	834 [734, 938]	−100 [−259, 30]
Transformers	Retrieval (ms)	138 [127, 149]	319 [307, 331]	+182 [166, 197]
	Generation (ms)	47,892 [44,298, 51,487]	44,005 [40,497, 47,487]	−3,887 [−6,995, −938]
	Prompt tokens	2,161 [2,031, 2,294]	2,323 [2,169, 2,480]	+162 [−6, 328]
	Completion tokens	330 [289, 370]	263 [224, 303]	−67 [−102, −33]
LangChain	Retrieval (ms)	105 [95, 116]	316 [300, 332]	+211 [192, 230]
	Generation (ms)	21,407 [19,297, 23,557]	19,499 [17,751, 21,302]	−1,908 [−3,815, −27]
	Prompt tokens	1,788 [1,599, 1,976]	1,607 [1,442, 1,774]	−181 [−380, 17]
	Completion tokens	65 [53, 77]	52 [44, 62]	−12 [−24, −1]

Hybrid retrieval adds a statistically positive mean retrieval overhead in every workload: 237 ms for LlamaIndex, 182 ms for Transformers, and 211 ms for LangChain. The LlamaIndex generation-

latency and token intervals include zero. In contrast, generation latency and completion tokens are lower for Hybrid-RAG on both new 100-case workloads, although prompt-token differences remain inconclusive. Local generation dominates end-to-end latency for both modes; the results support a sub-second mean retrieval claim on this machine, not a sub-second answer-response SLA.

The end-to-end user experience includes retrieval, context assembly, prompt processing, and local answer generation. Consequently, the measured retrieval target and the total time before a developer receives an answer are separate quantities: sub-second retrieval does not imply interactive end-to-end response time when mean local generation ranges from approximately 19 to 97 seconds across the measured workloads.

4.8 Case Study

To illustrate practical feasibility, we trace the benchmark query: *“What sequence does BaseRetriever.retrieve follow when it receives a plain query string, including callbacks and recursive retrieval?”*.

1. **Question analysis:** The query names `BaseRetriever.retrieve` and asks for an execution sequence involving callbacks and recursive retrieval.
2. **Retrieval channels:**
 - FalkorDB contributes structural relationships around the retriever implementation.
 - Qdrant retrieves semantically similar chunks from `base/base_retriever.py` and `retrievers/recursive_retriever.py`.
3. **RRF Rank Fusion:** The graph and vector ranked lists are fused before context assembly.
4. **Benchmark context selection:** The benchmark keeps the first five fused results and formats them directly as contexts. This case does not exercise the API’s optional dynamic token-budget parameters.
5. **Generation:** The answer explains the callback start, query-bundle conversion, retrieval, recursive resolution, callback completion, and return sequence. One case illustrates the path; aggregate quality is reported in Table 3.

5 Conclusion and Future Work

5.1 Conclusion

The completed experiments support a replicated but repository-specific conclusion:

1. **RQ1:** Hybrid-RAG has a statistically positive Faithfulness difference on Transformers and LangChain, and a statistically positive Answer Correctness difference on LlamaIndex and Transformers. Answer Relevancy is inconclusive in all three repositories.
2. **RQ2:** Mean retrieval latency increases by 182–237 ms across the workloads. Local generation remains dominant at approximately 19–97 seconds; the two 100-case workloads show lower Hybrid-RAG generation latency and completion-token counts, while the LlamaIndex intervals remain inconclusive.
3. **RQ3:** No quality metric has a positive paired 95% interval in all three repositories. The replication strengthens the evidence beyond one repository but does not establish pooled cross-repository, cross-language, or alternative-baseline superiority.

The principal technical contributions are:

- query-aware graph and vector retrieval with deterministic relation plans, weighted rank fusion, sibling expansion, and budgeted context assembly;
- repository identity scoping when supplied, replacement-before-delete indexing, provenance, cache-generation, and local-provider controls that make retrieval and evaluation runs auditable;
- deterministic directory-based communities with local Ollama-generated summaries, persisted dependencies, and repository-filtered global retrieval;
- a multi-repository 3D/2D visualization portal and bounded-concurrency indexing implementation; and
- reproducible benchmarks totaling 1,380 generated and scored records across three pinned repository workloads, with deterministic paired case-level confidence-interval analyses.

5.2 Limitations

- **Local Generation Latency:** Mean answer generation ranges from approximately 19 to 87 seconds for hybrid retrieval and 21 to 97 seconds for vector-only retrieval across the measured workloads; this is not a sub-second end-to-end system.
- **Language Constraints:** Semantic AST indexing currently supports Python only. Java projects are rejected explicitly, and optional non-code ingestion provides generic chunks rather than language-aware AST semantics.
- **Evaluation Scale:** The answer-quality evidence covers three pinned Python repository workloads with AI-reviewed reference answers, but LlamaIndex has 30 cases while Transformers and LangChain have 100 each. The corpus scopes also differ, so the estimates are reported separately and do not constitute a pooled benchmark. Independent human review is still required.
- **Baseline Coverage:** Only vector-only retrieval is executed under the paired protocol. BM25, graph-only retrieval, learned code encoders, and an independent GraphRAG implementation are not measured.
- **Cost Evidence:** Local Ollama produces no hosted API bill in this experiment, but the benchmark does not measure electricity, hardware depreciation, storage, or maintenance cost.
- **Statistical Power:** Significance varies by repository and metric, and no Answer Relevancy interval excludes zero. More independent questions and a matched cross-repository design are required before interpreting the estimates as universal improvements.

5.3 Future Work

- **Standardized Multi-Repository Replication:** Expand LlamaIndex to 100 cases, apply the same corpus-selection policy to all repositories, add further Python repositories, and report a pooled estimate only after those design differences are removed.
- **Additional Baselines and Ablations:** Execute BM25, graph-only, vector-only, Hybrid-RAG, and a documented external GraphRAG baseline under matched questions and gen-

eration settings.

- **Language Adapters:** Add complete entity extraction, relation resolution, chunking, and tests for Java and Go before using Java projects or Kubernetes as evaluation corpora.
- **Ambiguous Cross-Repository Resolution:** Extend the current unique-match resolver with import-aware namespace disambiguation, then evaluate cross-repository relation questions explicitly.
- **Human Review and Power Analysis:** Add independent human reference review and determine the number of cases required to narrow the confidence intervals for each quality metric.
- **Adaptive RRF Weights:** Tune graph and vector weights on a separate development set and evaluate them on untouched repositories to avoid test-set optimization.
- **Literature Refresh:** Add verified 2025–2026 references on code-RAG, repository-level retrieval, and agentic software engineering before the final report submission.

References

1. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
2. G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 758–759, 2009.
3. S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
4. D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, C. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
5. M. Brunsfeld, “Tree-sitter: An incremental parsing system for programming tools,” GitHub, 2018. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
6. N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
7. S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pp. 150–163, 2024.
8. M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Pinto de Oliveira, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
9. Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020.
10. R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, *et al.*, “StarCoder: May the source be with you!,” *Transactions on Machine Learning Research*, 2023.
11. D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, *et al.*, “DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence,” *arXiv preprint arXiv:2401.14196*, 2024.

12. S. Kalliamvakou, “Research: Quantifying GitHub Copilot’s impact on developer productivity and happiness,” GitHub Blog, 2022. [Online]. Available: <https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>
13. D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, *et al.*, “GraphCodeBERT: Pre-training code representations with data flow,” in *International Conference on Learning Representations (ICLR)*, 2021.
14. A. Cockburn, “Hexagonal architecture,” 2005. [Online]. Available: <https://alistair.cockburn.us/hexagonal-architecture/>
15. FalkorDB Contributors, “FalkorDB: Ultra-fast graph database,” 2024. [Online]. Available: <https://www.falkordb.com/>
16. Qdrant Team, “Qdrant: Vector similarity search engine,” 2024. [Online]. Available: <https://qdrant.tech/>
17. Z. Nussbaum, J. Morris, B. Duderstadt, and A. Mulyar, “Nomic Embed: Training a reproducible long context text embedder,” *arXiv preprint arXiv:2402.01613*, 2024.
18. Ollama Contributors, “Ollama: Run large language models locally,” 2024. [Online]. Available: <https://ollama.com/>
19. J. Liu, “LlamaIndex: Data framework for LLM applications,” 2024. [Online]. Available: <https://www.llamaindex.ai/>
20. C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494, 2023.
21. N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

Copyright Acknowledgements

Except where otherwise stated through citations and source acknowledgements, the text, diagrams, tables, and experimental analysis in this thesis were prepared by the author. Third-party publications, software, datasets, and online resources are identified in the References. Their inclusion is solely for academic discussion and does not transfer ownership of the original material.